

Next.js 15 Full-Stack Course

5-Day Modern Web Development Course

Table of Contents

1. [Course Overview](#)
2. [Day 1: Next.js Foundations - App Router & Server Components](#)
3. [Day 2: Routing & Navigation - Dynamic Routes, Layouts & Middleware](#)
4. [Day 3: Data Fetching, Caching & Rendering Strategies](#)
5. [Day 4: Server Actions, API Routes & Authentication](#)
6. [Day 5: Performance, Deployment & Production Readiness](#)
7. [Appendix: Answer Keys & Code Solutions](#)
8. [Appendix: Project Setup Guide](#)

Course Overview

What You Will Learn

By the end of this 5-day Next.js 15 course, you will be able to:

- Understand the App Router architecture and file-system routing
- Build pages with Server Components and Client Components
- Create dynamic routes, route groups, and advanced routing patterns
- Fetch data using Server Components with `async/await`
- Implement caching and revalidation strategies
- Use Server Actions for form mutations
- Build API Routes (Route Handlers)
- Implement authentication with middleware
- Optimize performance with streaming, Suspense, and ISR
- Deploy production-ready applications

Prerequisites

- Basic knowledge of React (components, hooks, state)
- Basic JavaScript/TypeScript

- Node.js 18+ installed
- VS Code or similar editor

Course Materials

- Computer with Node.js 18+
- VS Code with TypeScript and Tailwind extensions
- Git installed
- GitHub account (for deployment)
- Vercel account (optional but recommended)

Course Setup

Creating the Project

```
# Create a new Next.js 15 project with TypeScript and Tailwind
npx create-next-app@latest nextjs-course-app --typescript --tailwind --eslint --app --

# Navigate to project
cd nextjs-course-app

# Install additional dependencies
npm install @types/node @types/react @types/react-dom -D

# Start development server
npm run dev
```

Project Structure (App Router):

```
nextjs-course-app/
├── src/
│   └── app/
│       ├── globals.css
│       ├── layout.tsx      # Root layout (Server Component)
│       ├── page.tsx       # Home page (Server Component)
│       ├── (marketing)/   # Route group - no URL impact
│       │   ├── about/
│       │   │   └── page.tsx # /about
│       │   ├── contact/
│       │   │   └── page.tsx # /contact
│       ├── (dashboard)/   # Route group - different layout
│       │   ├── layout.tsx  # Dashboard layout with sidebar
│       │   └── dashboard/
```

```
├── page.tsx      # /dashboard
├── blog/
│   ├── page.tsx  # /blog
│   ├── [slug]/   # Dynamic route
│   └── page.tsx  # /blog/hello-world
├── api/          # API Routes
│   └── posts/
│       └── route.ts # /api/posts
├── loading.tsx   # Loading UI (Suspense fallback)
├── error.tsx     # Error UI
├── not-found.tsx # 404 UI
├── public/
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

Day 1: Next.js Foundations

App Router & Server Components

Learning Objectives

By the end of Day 1, you will be able to:

- Create a Next.js 15 project with TypeScript and Tailwind
- Understand the App Router file-system routing
- Create pages and layouts
- Understand Server Components vs Client Components
- Build your first Next.js pages

1.1 What is Next.js?

The Philosophy Behind Next.js

Next.js is a React framework that provides:

- **File-system routing:** Pages are automatically routed based on file structure
- **Server-side rendering (SSR):** Pages can be rendered on the server before being sent to the browser
- **Static site generation (SSG):** Pages can be pre-rendered at build time

- **API routes:** Build full-stack applications with API endpoints
- **Image optimization:** Automatic image optimization with next/image
- **Middleware:** Run code before a request is completed

Why Next.js?

Feature	Benefit
App Router	New routing architecture with nested layouts and Server Components
Server Components	Render React components on the server, reducing client-side JavaScript
Automatic code splitting	Only load what's needed for each page
SEO friendly	Pre-rendered pages are indexable by search engines
Fast refresh	Instant feedback during development

1.2 App Router File-System Routing

The Foundation: Files Become Routes

In the App Router, files determine routes:

```
app/
├─ page.tsx           → Route: /
├─ about/page.tsx     → Route: /about
├─ blog/page.tsx      → Route: /blog
└─ blog/[slug]/page.tsx → Route: /blog/hello-world
```

Key Files:

File	Purpose
page.tsx	Defines the page content for a route
layout.tsx	Shared UI wrapper for a route segment and its children
loading.tsx	Loading UI displayed during Suspense
error.tsx	Error UI for handling errors

File	Purpose
not-found.tsx	404 UI
route.ts	API route handler

Hands-On: Create Your First Pages

Step 1: Home Page - `app/page.tsx`

```
// src/app/page.tsx
export default function HomePage() {
  return (
    <main className="min-h-screen flex items-center justify-center">
      <div className="text-center">
        <h1 className="text-4xl font-bold">Welcome to Next.js 15!</h1>
        <p className="mt-4 text-gray-600">App Router with TypeScript and Tailwind</p>
      </div>
    </main>
  );
}
```

Step 2: About Page - `app/about/page.tsx`

```
// src/app/about/page.tsx
export default function AboutPage() {
  return (
    <main className="min-h-screen flex items-center justify-center">
      <div className="text-center">
        <h1 className="text-4xl font-bold">About Us</h1>
        <p className="mt-4 text-gray-600">Built with Next.js 15 App Router</p>
      </div>
    </main>
  );
}
```

Step 3: Blog Page - `app/blog/page.tsx`

```
// src/app/blog/page.tsx
export default function BlogPage() {
  return (
    <main className="min-h-screen p-8">
      <h1 className="text-4xl font-bold">Blog</h1>
      <ul className="mt-4 space-y-2">
        <li>📝 Blog Post 1</li>
      </ul>
    </main>
  );
}
```

```

    <li>📝 Blog Post 2</li>
    <li>📝 Blog Post 3</li>
  </ul>
</main>
);
}

```

Step 4: Dynamic Blog Post - `app/blog/[slug]/page.tsx`

```

// src/app/blog/[slug]/page.tsx

// Note: In Next.js 15, params is a Promise
export default async function BlogPostPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;

  return (
    <main className="min-h-screen p-8">
      <h1 className="text-4xl font-bold">Blog Post: {slug}</h1>
      <p className="mt-4 text-gray-600">This is the content for {slug}</p>
    </main>
  );
}

```

1.3 Layouts: The Shared UI

What is a Layout?

A layout is a wrapper component that shares UI across multiple pages.

Root Layout - `app/layout.tsx`

```

// src/app/layout.tsx
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";

const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: "My Next.js App",
  description: "Built with Next.js 15 and App Router",
}

```

```

};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body className={inter.className}>
        <header className="bg-gray-800 text-white p-4">
          <nav className="container mx-auto flex gap-4">
            <a href="/" className="hover:underline">Home</a>
            <a href="/about" className="hover:underline">About</a>
            <a href="/blog" className="hover:underline">Blog</a>
          </nav>
        </header>
        {children}
        <footer className="bg-gray-100 text-center p-4 mt-8">
          <p>© 2026 My Next.js App</p>
        </footer>
      </body>
    </html>
  );
}

```

Nested Layout - app/blog/layout.tsx

```

// src/app/blog/layout.tsx
export default function BlogLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="container mx-auto px-4 py-8">
      <aside className="bg-gray-50 p-4 rounded-lg mb-6">
        <p>📖 Blog Sidebar – Shared across blog pages</p>
      </aside>
      <main>{children}</main>
    </div>
  );
}

```

1.4 Server Components vs Client Components

Server Components (Default)

Server Components render on the server and send HTML to the browser.

Why use Server Components:

- **Less JavaScript:** No JavaScript sent to the client
- **Direct data access:** Can access databases directly
- **Better performance:** Faster page loads
- **SEO friendly:** HTML is pre-rendered

What Server Components CAN do:

- Fetch data with `async/await`
- Access databases
- Use Node.js APIs (`fs`, `path`, etc.)
- Use server-side environment variables

What Server Components CANNOT do:

- Use React hooks (`useState` , `useEffect`)
- Use browser APIs (`window` , `document`)
- Handle user interactions (clicks, input)

Client Components (use client)

Client Components render on the client and are interactive.

When to use Client Components:

- Interactive UI (buttons, forms, modals)
- React hooks (`useState` , `useEffect`)
- Browser APIs (`window` , `document`)
- Third-party libraries that need client-side rendering

Example: Counter Component

```
// src/components/Counter.tsx
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
```

```
    return (  
      <button  
        onClick={() => setCount(count + 1)}  
        className="px-4 py-2 bg-blue-500 text-white rounded"  
      >  
        Count: {count}  
      </button>  
    );  
  }  
}
```

Using Client Component in Server Component:

```
// src/app/page.tsx  
import Counter from '@components/Counter';  
  
export default function HomePage() {  
  return (  
    <main className="min-h-screen flex items-center justify-center">  
      <div className="text-center">  
        <h1 className="text-4xl font-bold">Welcome!</h1>  
        <div className="mt-8">  
          <Counter />  
        </div>  
      </div>  
    </main>  
  );  
}
```

When to Use Which

Use Server Components For...	Use Client Components For...
Pages that display data	Interactive UI elements
SEO-critical content	Forms and inputs
Static content	Components with state
Data fetching	Components with effects
Marketing pages	Modals, toggles, dropdowns

Rule of thumb: Make everything a Server Component by default. Add `'use client'` only when you need interactivity.

1.5 The Metadata API

Static Metadata

```
// app/about/page.tsx
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "About Us",
  description: "Learn more about our company and mission.",
  openGraph: {
    title: "About Us",
    description: "Learn about our team and mission.",
    type: "website",
  },
};

export default function AboutPage() {
  return <main>{/* Content */</main>;
}
```

Dynamic Metadata

For pages where metadata depends on data:

```
// app/blog/[slug]/page.tsx
import type { Metadata } from "next";

export async function generateMetadata({
  params,
}: {
  params: Promise<{ slug: string }>;
}): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPost(slug);

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
    },
  };
}

async function getPost(slug: string) {
  // Fetch post from database or API
  return { title: `Post: ${slug}`, excerpt: `Content for ${slug}` };
}
```

Day 1 Exercises

Exercise 1.1: Create Your First Pages

Instructions: Create the following pages in your Next.js app:

1. A contact page at `/contact`
2. A blog page at `/blog` with 5 hardcoded blog posts
3. A dynamic blog post page at `/blog/[slug]` that displays the slug

Solution:

```
// src/app/contact/page.tsx
export default function ContactPage() {
  return (
    <main className="min-h-screen p-8">
      <h1 className="text-4xl font-bold">Contact Us</h1>
      <p className="mt-4 text-gray-600">Get in touch with us</p>
      <form className="mt-8 max-w-md">
        <div className="mb-4">
          <label className="block text-sm font-medium mb-2">Name</label>
          <input type="text" className="w-full border rounded px-3 py-2" />
        </div>
        <div className="mb-4">
          <label className="block text-sm font-medium mb-2">Email</label>
          <input type="email" className="w-full border rounded px-3 py-2" />
        </div>
        <button className="bg-blue-500 text-white px-4 py-2 rounded">
          Send
        </button>
      </form>
    </main>
  );
}
```

```
// src/app/blog/page.tsx
const posts = [
  { slug: "hello-world", title: "Hello World" },
  { slug: "getting-started", title: "Getting Started with Next.js" },
  { slug: "app-router", title: "Understanding the App Router" },
  { slug: "server-components", title: "Server Components Explained" },
  { slug: "deployment", title: "Deploying Your Next.js App" },
];

export default function BlogPage() {
  return (
    <main className="min-h-screen p-8">
```

```

    <h1 className="text-4xl font-bold">Blog</h1>
    <ul className="mt-4 space-y-2">
      {posts.map((post) => (
        <li key={post.slug}>
          <a href={`/blog/${post.slug}`} className="text-blue-500 hover:underline">
            📄 {post.title}
          </a>
        </li>
      ))}
    </ul>
  </main>
);
,

```

```
// src/app/blog/[slug]/page.tsx
```

```

const posts = [
  { slug: "hello-world", title: "Hello World" },
  { slug: "getting-started", title: "Getting Started with Next.js" },
  { slug: "app-router", title: "Understanding the App Router" },
  { slug: "server-components", title: "Server Components Explained" },
  { slug: "deployment", title: "Deploying Your Next.js App" },
];

```

```

export default async function BlogPostPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;
  const post = posts.find((p) => p.slug === slug);

  if (!post) {
    return (
      <main className="min-h-screen p-8">
        <h1 className="text-4xl font-bold">404 - Post Not Found</h1>
        <p className="mt-4">The post "{slug}" does not exist.</p>
      </main>
    );
  }

  return (
    <main className="min-h-screen p-8">
      <h1 className="text-4xl font-bold">{post.title}</h1>
      <p className="mt-4 text-gray-600">This is the full content for {post.slug}</p>
      <a href="/blog" className="mt-8 inline-block text-blue-500 hover:underline">
        ← Back to Blog
      </a>
    </main>
  );
}

```

```
);
}
```

Exercise 1.2: Create a Layout with Navigation

Instructions: Create a root layout with a navigation bar and footer. Then create a blog layout with a sidebar.

Root Layout Solution:

```
// src/app/layout.tsx
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "../globals.css";

const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: "Next.js 15 Course",
  description: "Learning Next.js 15 with App Router",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body className={inter.className}>
        <!-- Navigation -->
        <nav className="bg-gray-800 text-white p-4">
          <div className="container mx-auto flex items-center justify-between">
            <a href="/" className="text-xl font-bold">Next.js Course</a>
            <div className="flex gap-6">
              <a href="/" className="hover:text-gray-300">Home</a>
              <a href="/about" className="hover:text-gray-300">About</a>
              <a href="/blog" className="hover:text-gray-300">Blog</a>
              <a href="/contact" className="hover:text-gray-300">Contact</a>
            </div>
          </div>
        </nav>

        <!-- Page Content -->
        <main>{children}</main>

        <!-- Footer -->
        <footer className="bg-gray-100 text-center p-4 mt-8 border-t">
```

```

    <p className="text-sm text-gray-600">© 2026 Next.js 15 Course</p>
  </footer>
</body>
</html>
);
}

```

Blog Layout Solution:

```

// src/app/blog/layout.tsx
export default function BlogLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="container mx-auto px-4 py-8">
      <div className="grid grid-cols-1 md:grid-cols-4 gap-8">
        {/* Sidebar */}
        <aside className="md:col-span-1">
          <div className="bg-gray-50 p-4 rounded-lg sticky top-4">
            <h3 className="font-bold text-lg mb-4">Blog</h3>
            <ul className="space-y-2 text-gray-600">
              <li><a href="/blog" className="hover:text-blue-500">All Posts</a></li>
              <li><a href="#" className="hover:text-blue-500">Categories</a></li>
              <li><a href="#" className="hover:text-blue-500">Archive</a></li>
            </ul>
          </div>
        </aside>

        {/* Main Content */}
        <main className="md:col-span-3">{children}</main>
      </div>
    </div>
  );
}

```

Exercise 1.3: Create a Client Component

Instructions: Create a counter component with `useState` and use it on the home page.

Counter Component Solution:

```

// src/components/Counter.tsx
'use client';

```

```
import { useState } from 'react';

interface CounterProps {
  initialCount?: number;
}

export default function Counter({ initialCount = 0 }: CounterProps) {
  const [count, setCount] = useState(initialCount);

  return (
    <div className="flex flex-col items-center gap-4 p-6 bg-gray-50 rounded-lg">
      <div className="text-4xl font-bold">{count}</div>
      <div className="flex gap-4">
        <button
          onClick={() => setCount(count - 1)}
          className="px-4 py-2 bg-red-500 text-white rounded hover:bg-red-600"
        >
          -1
        </button>
        <button
          onClick={() => setCount(count + 1)}
          className="px-4 py-2 bg-blue-500 text-white rounded hover:bg-blue-600"
        >
          +1
        </button>
        <button
          onClick={() => setCount(0)}
          className="px-4 py-2 bg-gray-500 text-white rounded hover:bg-gray-600"
        >
          Reset
        </button>
      </div>
    </div>
  );
}
```

Exercise 1.4: Add Metadata

Instructions: Add static metadata to the about page and dynamic metadata to the blog post page.

About Page Metadata:

```
// src/app/about/page.tsx
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "About Us | Next.js 15 Course",
  description: "Learn about the Next.js 15 course and what you'll build.",
};
```

```
export default function AboutPage() {  
  return (  
    <main className="min-h-screen p-8">  
      <h1 className="text-4xl font-bold">About Us</h1>  
      <p className="mt-4 text-gray-600">This is the about page.</p>  
    </main>  
  );  
}
```

Day 1 Summary

Key Takeaways

1. **App Router:** Files determine routes. `page.tsx` creates pages, `layout.tsx` creates shared UI.
2. **Server Components:** Default. Render on server, no JavaScript sent to client.
3. **Client Components:** Use `'use client'` directive. For interactive UI with hooks.
4. **Metadata:** Use `metadata` export for static SEO or `generateMetadata` for dynamic SEO.
5. **Rule of thumb:** Server Component by default. Only use `'use client'` when necessary.

Quiz Yourself

1. What file creates a page in the App Router?
2. What is the difference between Server Components and Client Components?
3. When should you use `'use client'` ?
4. What is a layout?
5. How do you add dynamic metadata?

Day 2: Routing & Navigation

Dynamic Routes, Layouts & Middleware

Learning Objectives

By the end of Day 2, you will be able to:

- Create dynamic routes with `[slug]`
- Use route groups `(group)` for organization
- Implement middleware for authentication

- Navigate between pages with Link and useRouter
- Handle loading and error states

2.1 Dynamic Routes

Basic Dynamic Routes

Create dynamic routes using square brackets [param] :

```

app/
├── blog/
│   ├── [slug]/
│   │   └── page.tsx    → /blog/hello-world
├── products/
│   ├── [id]/
│   │   └── page.tsx    → /products/123
└── posts/
    ├── [category]/
    │   ├── [slug]/
    │   │   └── page.tsx → /posts/tech/hello-world
    └── page.tsx
  
```

Example: Dynamic Route with Data

```

// app/blog/[slug]/page.tsx

// Define the type for params (Next.js 15 uses Promise)
interface BlogPostPageProps {
  params: Promise<{ slug: string }>;
}

async function getPost(slug: string) {
  // In real app, fetch from database or API
  const posts = {
    "hello-world": {
      title: "Hello World",
      content: "This is the first blog post.",
      date: "2026-01-01",
    },
    "app-router": {
      title: "App Router Guide",
      content: "Learn about the App Router.",
      date: "2026-01-02",
    },
  };
};
return posts[slug as keyof typeof posts] || null;
  
```

```

    }

    export default async function BlogPostPage({ params }: BlogPostPageProps) {
      const { slug } = await params;
      const post = await getPost(slug);

      if (!post) {
        return (
          <div className="p-8">
            <h1 className="text-2xl font-bold">Post not found</h1>
            <p className="mt-2">The post "{slug}" does not exist.</p>
          </div>
        );
      }

      return (
        <div className="p-8 max-w-2xl">
          <h1 className="text-3xl font-bold">{post.title}</h1>
          <p className="text-gray-500 text-sm mt-1">{post.date}</p>
          <p className="mt-4">{post.content}</p>
        </div>
      );
    }
  }
}

```

Static Generation with generateStaticParams

For pages that should be generated at build time:

```

// app/blog/[slug]/page.tsx

export async function generateStaticParams() {
  // This function runs at build time
  const posts = [
    { slug: "hello-world" },
    { slug: "app-router" },
    { slug: "server-components" },
  ];

  return posts.map((post) => ({
    slug: post.slug,
  }));
}

// The page component remains the same
export default async function BlogPostPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;

```

```
// ... rest of the component
}
```

2.2 Route Groups

What Are Route Groups?

Route groups organize routes without affecting the URL path. They use parentheses: (group) .

```
app/
├── (marketing)/          # Group - no URL impact
│   ├── layout.tsx       # Marketing layout (no sidebar, full width)
│   ├── page.tsx         # / (homepage)
│   └── about/page.tsx   # /about
├── (dashboard)/         # Group - different layout
│   ├── layout.tsx       # Dashboard layout (with sidebar, auth required)
│   ├── dashboard/page.tsx # /dashboard
│   └── settings/page.tsx # /settings
└── (auth)/              # Group - auth flow
    ├── layout.tsx       # Auth layout (no header, centered forms)
    ├── login/page.tsx   # /login
    └── register/page.tsx # /register
```

Example: Different Layouts for Different Groups

Marketing Layout - `app/(marketing)/layout.tsx`

```
export default function MarketingLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div>
      <header className="bg-white shadow-sm p-4">
        <div className="container mx-auto flex justify-between">
          <a href="/" className="text-xl font-bold">MyApp</a>
          <nav className="flex gap-6">
            <a href="/about">About</a>
            <a href="/contact">Contact</a>
            <a href="/login" className="text-blue-500">Login</a>
          </nav>
        </div>
      </header>
      <main>{children}</main>
    </div>
  )
}
```

```
    </div>
  );
}
```

Dashboard Layout - `app/(dashboard)/layout.tsx`

```
export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="flex min-h-screen">
      {/* Sidebar */}
      <aside className="w-64 bg-gray-800 text-white p-6">
        <h2 className="text-xl font-bold mb-6">Dashboard</h2>
        <nav className="space-y-4">
          <a href="/dashboard" className="block hover:text-gray-300"><img alt="Bar chart icon" data-bbox="781 358 796 373"/> Overview</a>
          <a href="/dashboard/analytics" className="block hover:text-gray-300"><img alt="Line chart icon" data-bbox="881 376 896 391"/> Anal
          <a href="/dashboard/settings" className="block hover:text-gray-300"><img alt="Gear icon" data-bbox="871 394 886 409"/> Setti
          <a href="/" className="block hover:text-gray-300 mt-8"><img alt="Home icon" data-bbox="781 412 796 427"/> Back to Home</a>
        </nav>
      </aside>

      {/* Main Content */}
      <main className="flex-1 p-8 bg-gray-50">{children}</main>
    </div>
  );
}
```



Use Cases for Route Groups

Use Case	Example
Different layouts	Marketing vs. Dashboard
Authentication flow	Login/Register pages
Internationalization	(en) , (es) , (fr)
Feature modules	(shop) , (admin) , (api)

2.3 Navigation

The Link Component

Use `Link` instead of `<a>` for client-side navigation:

```
import Link from "next/link";

export default function Navigation() {
  return (
    <nav className="flex gap-6">
      <Link href="/" className="hover:underline">Home</Link>
      <Link href="/about" className="hover:underline">About</Link>
      <Link href="/blog" className="hover:underline">Blog</Link>
      <Link href="/contact" className="hover:underline">Contact</Link>
    </nav>
  );
}
```

Dynamic Links

```
// app/blog/page.tsx
import Link from "next/link";

const posts = [
  { slug: "hello-world", title: "Hello World" },
  { slug: "app-router", title: "App Router Guide" },
];

export default function BlogPage() {
  return (
    <div>
      <h1 className="text-3xl font-bold">Blog</h1>
      <ul className="mt-4 space-y-2">
        {posts.map((post) => (
          <li key={post.slug}>
            <Link href={`/blog/${post.slug}`} className="text-blue-500 hover:underline">
              {post.title}
            </Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

The useRouter Hook

For client-side navigation with programmatic control:

```
'use client';

import { useRouter } from "next/navigation";

export default function BackButton() {
  const router = useRouter();

  return (
    <button
      onClick={() => router.back()}
      className="text-blue-500 hover:underline"
    >
      ← Go Back
    </button>
  );
}
```

usePathname for Active Links

```
'use client';

import { usePathname } from "next/navigation";
import Link from "next/link";

export default function NavLinks() {
  const pathname = usePathname();

  const links = [
    { href: "/", label: "Home" },
    { href: "/about", label: "About" },
    { href: "/blog", label: "Blog" },
  ];

  return (
    <nav className="flex gap-6">
      {links.map((link) => (
        <Link
          key={link.href}
          href={link.href}
          className={`hover:underline ${
            pathname === link.href ? "font-bold text-blue-500" : ""
          }`}
        >
          {link.label}
        </Link>
      ))}
    </nav>
  );
}
```

2.4 Middleware

What is Middleware?

Middleware runs code before a request is completed. Common uses:

- Authentication
- Redirects
- A/B testing
- Logging
- Headers

Creating Middleware

Create `middleware.ts` at the root of your project (outside the `app` folder):

```
// middleware.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  // Check if user is authenticated (example)
  const token = request.cookies.get("auth-token");

  // Protected routes
  const isProtectedRoute = request.nextUrl.pathname.startsWith("/dashboard");

  if (isProtectedRoute && !token) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  // Continue to the requested page
  return NextResponse.next();
}

// Configure which routes middleware runs on
export const config = {
  matcher: [
    "/dashboard/:path*",
    "/settings/:path*",
    "/profile/:path*",
  ],
};
```

Advanced Middleware Example

```
// middleware.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const path = request.nextUrl.pathname;
  const token = request.cookies.get("auth-token");

  // Define protected and public routes
  const isProtectedRoute = path.startsWith("/dashboard") ||
    path.startsWith("/profile") ||
    path.startsWith("/settings");
  const isAuthRoute = path.startsWith("/login") || path.startsWith("/register");
  const isPublicRoute = path === "/" || path.startsWith("/about") || path.startsWith("/");

  // Redirect: Authenticated user tries to access auth routes
  if (isAuthRoute && token) {
    return NextResponse.redirect(new URL("/dashboard", request.url));
  }

  // Redirect: Unauthenticated user tries to access protected routes
  if (isProtectedRoute && !token) {
    const loginUrl = new URL("/login", request.url);
    loginUrl.searchParams.set("redirect", path);
    return NextResponse.redirect(loginUrl);
  }

  // Add custom header
  const response = NextResponse.next();
  response.headers.set("x-middleware-cache", "no-cache");

  return response;
}

export const config = {
  matcher: [
    /*
     * Match all request paths except:
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico (favicon file)
     * - public folder
     */
    "/((?!api|_next/static|_next/image|favicon.ico|public/).*)",
  ],
};
```

Loading UI

```
// app/blog/loading.tsx
export default function BlogLoading() {
  return (
    <div className="flex items-center justify-center min-h-[400px]">
      <div className="text-center">
        <div className="w-12 h-12 border-4 border-blue-500 border-t-transparent rounded">
          <p className="mt-4 text-gray-600">Loading blog posts...</p>
        </div>
      </div>
    </div>
  );
}
```



```
// app/blog/error.tsx
'use client';

import { useEffect } from "react";

interface ErrorProps {
  error: Error & { digest?: string };
  reset: () => void;
}

export default function BlogError({ error, reset }: ErrorProps) {
  useEffect(() => {
    console.error("Blog page error:", error);
  }, [error]);

  return (
    <div className="flex items-center justify-center min-h-[400px]">
      <div className="text-center">
        <h2 className="text-2xl font-bold text-red-600">Something went wrong!</h2>
        <p className="mt-2 text-gray-600">{error.message}</p>
        <button
          onClick={reset}
          className="mt-4 px-4 py-2 bg-blue-500 text-white rounded hover:bg-blue-600"
        >
```

```

        Try again
      </button>
    </div>
  </div>
);

```

Not Found UI

Create `not-found.tsx` for 404 pages:

```

// app/not-found.tsx
import Link from "next/link";

export default function NotFound() {
  return (
    <div className="flex items-center justify-center min-h-[400px]">
      <div className="text-center">
        <h1 className="text-6xl font-bold text-gray-300">404</h1>
        <h2 className="mt-2 text-2xl font-semibold">Page Not Found</h2>
        <p className="mt-2 text-gray-600">The page you're looking for doesn't exist.</p>
        <Link
          href="/"
          className="mt-6 inline-block px-4 py-2 bg-blue-500 text-white rounded hover:
        >
          Go Home
        </Link>
      </div>
    </div>
  );
}

```

Day 2 Exercises

Exercise 2.1: Create a Dynamic Product Page

Instructions:

1. Create a products page at `/products` with a list of products
2. Create a dynamic product page at `/products/[id]` that displays product details
3. Use `generateStaticParams` to pre-render products at build time

```
// app/products/page.tsx
import Link from "next/link";

const products = [
  { id: "1", name: "Product 1", price: "$19.99", description: "A great product" },
  { id: "2", name: "Product 2", price: "$29.99", description: "An even better product" },
  { id: "3", name: "Product 3", price: "$39.99", description: "The best product" },
];

export default function ProductsPage() {
  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Products</h1>
      <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mt-6">
        {products.map((product) => (
          <Link
            key={product.id}
            href={`\`/products/${product.id}\`}`
            className="border rounded-lg p-6 hover:shadow-lg transition"
          >
            <h2 className="text-xl font-semibold">{product.name}</h2>
            <p className="text-gray-600 mt-2">{product.description}</p>
            <p className="text-lg font-bold mt-4 text-blue-500">{product.price}</p>
          </Link>
        ))}
      </div>
    </div>
  );
}
```

```
// app/products/[id]/page.tsx

interface ProductPageProps {
  params: Promise<{ id: string }>;
}

const products = [
  { id: "1", name: "Product 1", price: "$19.99", description: "A great product" },
  { id: "2", name: "Product 2", price: "$29.99", description: "An even better product" },
  { id: "3", name: "Product 3", price: "$39.99", description: "The best product" },
];

export async function generateStaticParams() {
  return products.map((product) => ({
    id: product.id,
  }));
}

export default async function ProductPage({ params }: ProductPageProps) {
```

```

const { id } = await params;
const product = products.find((p) => p.id === id);

if (!product) {
  return (
    <div className="p-8">
      <h1 className="text-2xl font-bold">Product not found</h1>
      <p className="mt-2">Product ID "{id}" does not exist.</p>
    </div>
  );
}

return (
  <div className="p-8 max-w-2xl">
    <a href="/products" className="text-blue-500 hover:underline">← Back to Products
    <h1 className="text-3xl font-bold mt-4">{product.name}</h1>
    <p className="text-xl font-bold mt-2 text-blue-500">{product.price}</p>
    <p className="mt-4 text-gray-600">{product.description}</p>
  </div>
);

```

Exercise 2.2: Create Route Groups

Instructions:

1. Create a marketing group (marketing) with a homepage and about page
2. Create a dashboard group (dashboard) with a dashboard page
3. Create an auth group (auth) with login and register pages

File Structure:

```

app/
├── (marketing)/
│   ├── layout.tsx
│   ├── page.tsx          # /
│   └── about/page.tsx    # /about
├── (dashboard)/
│   ├── layout.tsx
│   └── dashboard/page.tsx # /dashboard
├── (auth)/
│   ├── layout.tsx
│   ├── login/page.tsx    # /login
│   └── register/page.tsx # /register
└── layout.tsx            # Root layout

```

Exercise 2.3: Implement Middleware

Instructions:

1. Create middleware that protects `/dashboard/*` routes
2. Redirect unauthenticated users to `/login`
3. Redirect authenticated users away from `/login` and `/register`

```
// middleware.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const path = request.nextUrl.pathname;
  const token = request.cookies.get("auth-token");

  const isProtectedRoute = path.startsWith("/dashboard");
  const isAuthRoute = path.startsWith("/login") || path.startsWith("/register");

  // Redirect authenticated users away from auth routes
  if (isAuthRoute && token) {
    return NextResponse.redirect(new URL("/dashboard", request.url));
  }

  // Redirect unauthenticated users away from protected routes
  if (isProtectedRoute && !token) {
    const loginUrl = new URL("/login", request.url);
    loginUrl.searchParams.set("redirect", path);
    return NextResponse.redirect(loginUrl);
  }

  return NextResponse.next();
}

export const config = {
  matcher: [
    "/dashboard/:path*",
    "/login",
    "/register",
  ],
};
```

Exercise 2.4: Loading and Error States

Instructions:

1. Add a `loading.tsx` to the products page
2. Add an `error.tsx` to the products page

3. Add a `not-found.tsx` to the root

Day 2 Summary

Key Takeaways

1. **Dynamic Routes:** Use `[slug]` for dynamic parameters. `params` is a Promise in Next.js 15.
2. **Route Groups:** Use `(group)` to organize routes without affecting URLs.
3. **Middleware:** Runs before requests. Use for authentication, redirects, logging.
4. **Navigation:** Use `Link` for client-side navigation, `useRouter` for programmatic navigation.
5. **Loading/Error:** Special files `loading.tsx`, `error.tsx`, `not-found.tsx` for UI states.

Day 3: Data Fetching, Caching & Rendering

Learning Objectives

By the end of Day 3, you will be able to:

- Fetch data in Server Components with `async/await`
- Use caching and revalidation strategies
- Understand rendering strategies (SSR, SSG, ISR)
- Implement Streaming with `Suspense`
- Use Parallel Routes

3.1 Data Fetching in Server Components

Async Components

Server Components can be async functions:

```
// app/blog/page.tsx
async function getPosts() {
  // In real app, fetch from database or API
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");
  return res.json();
}

export default async function BlogPage() {
  const posts = await getPosts();
```

```

return (
  <div className="p-8">
    <h1 className="text-3xl font-bold">Blog Posts</h1>
    <ul className="mt-4 space-y-2">
      {posts.map((post: any) => (
        <li key={post.id}>
          <h2 className="text-xl font-semibold">{post.title}</h2>
          <p className="text-gray-600">{post.body}</p>
        </li>
      ))}
    </ul>
  </div>
);
}

```

Direct Database Access

Since Server Components run on the server, you can query databases directly:

```

// app/dashboard/page.tsx
import { db } from "@lib/database";

export default async function DashboardPage() {
  const users = await db.query("SELECT * FROM users LIMIT 10");
  const orders = await db.query("SELECT * FROM orders LIMIT 10");

  return (
    <div>
      <h1>Dashboard</h1>
      <div>
        <h2>Users ({users.length})</h2>
        {/* Display users */}
      </div>
      <div>
        <h2>Orders ({orders.length})</h2>
        {/* Display orders */}
      </div>
    </div>
  );
}

```

3.2 Caching and Revalidation

The fetch API with Caching

```
// Static data - cached indefinitely
const staticData = await fetch("https://api.example.com/static", {
  cache: "force-cache",
});

// Dynamic data - fetched on every request
const dynamicData = await fetch("https://api.example.com/live", {
  cache: "no-store",
});

// Revalidate after 60 seconds
const revalidatedData = await fetch("https://api.example.com/data", {
  next: {
    revalidate: 60, // 60 seconds
  },
});
```

Revalidation Options

Option	Behavior	Use Case
cache: "force-cache"	Cache indefinitely	Static data
cache: "no-store"	Fetch fresh every time	Real-time data
revalidate: 60	Cache for 60s, then revalidate	Semi-dynamic data

Revalidation with Tags

For more granular revalidation:

```
// app/posts/page.tsx
export default async function PostsPage() {
  const posts = await fetch("https://api.example.com/posts", {
    next: {
      tags: ["posts"], // Tag for revalidation
    },
  });
}

// ...
}

// app/api/revalidate/route.ts
import { revalidateTag } from "next/cache";
import { NextResponse } from "next/server";
```

```
export async function POST() {  
  // Revalidate all posts  
  revalidateTag("posts");  
  return NextResponse.json({ success: true });  
}
```

3.3 Rendering Strategies

Static Site Generation (SSG)

Pages are rendered at build time.

```
// app/posts/page.tsx  
export const dynamic = "force-static"; // Force static generation  
  
// OR for dynamic routes  
export async function generateStaticParams() {  
  const posts = await getPosts();  
  return posts.map((post: any) => ({  
    id: post.id,  
  }));  
}
```

Dynamic Rendering (SSR)

Pages are rendered on each request.

```
// app/dashboard/page.tsx  
export const dynamic = "force-dynamic"; // Force dynamic rendering  
  
// OR use cache: "no-store"  
const data = await fetch("https://api.example.com/live", {  
  cache: "no-store",  
});
```

Incremental Static Regeneration (ISR)

Pages are static but revalidate after a set time.

```
// app/products/page.tsx  
export const revalidate = 3600; // Revalidate every hour  
  
// OR in fetch  
const products = await fetch("https://api.example.com/products", {
```

```
    next: {
      revalidate: 3600,
    },
  });
```

Route Segment Config

```
// app/products/page.tsx

// Force static generation
export const dynamic = "force-static";

// Force dynamic rendering
export const dynamic = "force-dynamic";

// Revalidate every hour
export const revalidate = 3600;
```

3.4 Streaming with Suspense

What is Streaming?

Streaming sends HTML in chunks as data becomes available, improving perceived performance.

Basic Streaming with Suspense

```
// app/dashboard/page.tsx
import { Suspense } from "react";
import { UserList } from "../user-list";
import { OrderList } from "../order-list";

export default function DashboardPage() {
  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Dashboard</h1>

      {/* Each section loads independently */}
      <div className="grid grid-cols-2 gap-8 mt-8">
        <Suspense fallback=<div>Loading users...</div>>
          <UserList />
        </Suspense>

        <Suspense fallback=<div>Loading orders...</div>>
          <OrderList />
        </Suspense>
      </div>
    </div>
  );
}
```

```

    </Suspense>
  </div>
</div>
);
}

```

UserList Component:

```

// app/dashboard/user-list.tsx
async function getUsers() {
  const res = await fetch("https://jsonplaceholder.typicode.com/users");
  return res.json();
}

export default async function UserList() {
  const users = await getUsers();

  return (
    <div className="border rounded-lg p-4">
      <h2 className="text-xl font-bold">Users ({users.length})</h2>
      <ul className="mt-2 space-y-1">
        {users.map((user: any) => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
  );
}

```

OrderList Component:

```

// app/dashboard/order-list.tsx
async function getOrders() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");
  return res.json();
}

export default async function OrderList() {
  const orders = await getOrders();

  return (
    <div className="border rounded-lg p-4">
      <h2 className="text-xl font-bold">Orders ({orders.length})</h2>
      <ul className="mt-2 space-y-1">
        {orders.slice(0, 5).map((order: any) => (
          <li key={order.id}>{order.title}</li>
        ))}
      </ul>
    </div>
  );
}

```

```
);
}
```

3.5 Parallel Routes

What are Parallel Routes?

Parallel Routes render multiple pages in the same layout simultaneously.

```
app/
├── dashboard/
│   ├── layout.tsx
│   ├── page.tsx
│   ├── @analytics/      # Slot
│   │   └── page.tsx
│   └── @metrics/        # Slot
│       └── page.tsx
```

Dashboard Layout with Slots

```
// app/dashboard/layout.tsx
interface DashboardLayoutProps {
  children: React.ReactNode;
  analytics: React.ReactNode; // @analytics slot
  metrics: React.ReactNode;   // @metrics slot
}

export default function DashboardLayout({
  children,
  analytics,
  metrics,
}: DashboardLayoutProps) {
  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Dashboard</h1>
      <div className="grid grid-cols-3 gap-8 mt-8">
        <div className="col-span-2">{children}</div>
        <div className="space-y-8">
          <div>{analytics}</div>
          <div>{metrics}</div>
        </div>
      </div>
    </div>
  );
}
```

```
);
}
```

Slot Pages

```
// app/dashboard/@analytics/page.tsx
export default function Analytics() {
  return (
    <div className="border rounded-lg p-4">
      <h2 className="text-xl font-bold">📊 Analytics</h2>
      <p className="mt-2 text-gray-600">Analytics data goes here</p>
    </div>
  );
}
```

```
// app/dashboard/@metrics/page.tsx
export default function Metrics() {
  return (
    <div className="border rounded-lg p-4">
      <h2 className="text-xl font-bold">📈 Metrics</h2>
      <p className="mt-2 text-gray-600">Metrics data goes here</p>
    </div>
  );
}
```

Parallel Routes with Loading

Each slot can have its own loading state:

```
app/dashboard/
├── @analytics/
│   ├── page.tsx
│   └── loading.tsx    # Loading for analytics
└── @metrics/
    ├── page.tsx
    └── loading.tsx    # Loading for metrics
```

Day 3 Exercises

Exercise 3.1: Fetch Data in Server Components

Instructions:

1. Create a posts page that fetches data from an API
2. Display the posts with title and body

```
// app/posts/page.tsx
async function getPosts() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");
  return res.json();
}

export default async function PostsPage() {
  const posts = await getPosts();

  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Posts</h1>
      <div className="grid gap-4 mt-6">
        {posts.map((post: any) => (
          <div key={post.id} className="border rounded-lg p-4">
            <h2 className="text-xl font-semibold">{post.title}</h2>
            <p className="text-gray-600">{post.body}</p>
          </div>
        ))}
      </div>
    </div>
  );
}
```

Exercise 3.2: Implement Caching Strategies

Instructions:

1. Create three API endpoints with different caching strategies
2. Use force-cache, no-store, and revalidate

```
// app/demo/page.tsx
export default async function DemoPage() {
  // Static data - cached
  const staticData = await fetch("https://api.example.com/static", {
    cache: "force-cache",
  });

  // Dynamic data - fresh every request
  const dynamicData = await fetch("https://api.example.com/dynamic", {
    cache: "no-store",
  });

  // Revalidated data - every 60 seconds
  const revalidatedData = await fetch("https://api.example.com/revalidated", {
```

```

    next: {
      revalidate: 60,
    },
  });

  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Caching Demo</h1>
      { /* Display data */ }
    </div>
  );
}

```

Exercise 3.3: Implement Streaming

Instructions:

1. Create a dashboard with two sections that load independently
2. Use Suspense with fallback
3. Each section should show a loading state

```

// app/streaming-demo/page.tsx
import { Suspense } from "react";

// Simulated async component
async function SlowComponent() {
  await new Promise((resolve) => setTimeout(resolve, 2000));
  return <div>Loaded after 2 seconds!</div>;
}

export default function StreamingDemo() {
  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Streaming Demo</h1>
      <Suspense fallback={<div>Loading slow component...</div>}>
        <SlowComponent />
      </Suspense>
    </div>
  );
}

```

Exercise 3.4: Parallel Routes

Instructions:

1. Create a dashboard with parallel routes

2. Use `@analytics` and `@metrics` slots
3. Each slot should have independent loading

Day 3 Summary

Key Takeaways

1. **Data Fetching:** Server Components can be async and fetch data directly.
2. **Caching:** Use `cache: "force-cache"`, `cache: "no-store"`, or `revalidate: N`.
3. **Rendering Strategies:** SSG, SSR, ISR. Use `dynamic` and `revalidate` exports.
4. **Streaming:** Use `Suspense` to show loading states for each component independently.
5. **Parallel Routes:** Use slots `@folder` to render multiple pages in the same layout.

Day 4: Server Actions, API Routes & Authentication

Learning Objectives

By the end of Day 4, you will be able to:

- Build Server Actions for form mutations
- Create API Routes (Route Handlers)
- Implement authentication with NextAuth/Auth.js
- Use middleware for route protection
- Handle form validation with Zod

4.1 Server Actions

What Are Server Actions?

Server Actions are functions that run on the server, called from Client Components or forms. They're type-safe and automatically handle revalidation.

Basic Server Action

```
// app/actions/user.ts
'use server';

import { revalidatePath } from "next/cache";

export async function updateProfile(formData: FormData) {
  const name = formData.get("name");
  const email = formData.get("email");

  // Update database (example)
  console.log(`Updating profile: ${name}, ${email}`);

  // Revalidate the page to show updated data
  revalidatePath("/profile");
}
```

Using Server Actions in Forms

```
// app/profile/page.tsx
import { updateProfile } from "@app/actions/user";

export default function ProfilePage() {
  return (
    <div className="p-8 max-w-md">
      <h1 className="text-2xl font-bold mb-6">Update Profile</h1>

      <form action={updateProfile} className="space-y-4">
        <div>
          <label className="block text-sm font-medium mb-1">Name</label>
          <input
            type="text"
            name="name"
            className="w-full border rounded px-3 py-2"
            defaultValue="John Doe"
          />
        </div>

        <div>
          <label className="block text-sm font-medium mb-1">Email</label>
          <input
            type="email"
            name="email"
            className="w-full border rounded px-3 py-2"
            defaultValue="john@example.com"
          />
        </div>

        <button
          type="submit"
          className="w-full bg-blue-500 text-white py-2 rounded hover:bg-blue-600"
        >

```

```

    >
      Save Changes
    </button>
  </form>
</div>
);
}

```

Server Action with Validation (Zod)

```

// app/actions/user.ts
'use server';

import { revalidatePath } from "next/cache";
import { z } from "zod";

const userSchema = z.object({
  name: z.string().min(1, "Name is required"),
  email: z.string().email("Invalid email address"),
  age: z.number().min(18, "Must be 18 or older"),
});

interface FormState {
  success?: boolean;
  error?: string;
  errors?: {
    name?: string;
    email?: string;
    age?: string;
  };
};

export async function updateProfile(
  prevState: FormState,
  formData: FormData
): Promise<FormState> {
  try {
    const validated = userSchema.parse({
      name: formData.get("name"),
      email: formData.get("email"),
      age: Number(formData.get("age")),
    });

    // Update database...
    console.log("Updating user:", validated);

    revalidatePath("/profile");

    return { success: true };
  } catch (error) {
    if (error instanceof z.ZodError) {

```

```

    const fieldErrors: Record<string, string> = {};
    error.errors.forEach((err) => {
      if (err.path[0]) {
        fieldErrors[err.path[0] as string] = err.message;
      }
    });
    return { success: false, errors: fieldErrors };
  }

  return { success: false, error: "Something went wrong" };
}
}

```

Using useFormState

```

// app/profile/page.tsx
'use client';

import { useFormState } from "react-dom";
import { updateProfile } from "@app/actions/user";

const initialState = {
  success: false,
  error: "",
  errors: {},
};

export default function ProfilePage() {
  const [state, formAction] = useFormState(updateProfile, initialState);

  return (
    <div className="p-8 max-w-md">
      <h1 className="text-2xl font-bold mb-6">Update Profile</h1>

      {state.success && (
        <div className="mb-4 p-3 bg-green-100 text-green-700 rounded">
          Profile updated successfully!
        </div>
      )}

      {state.error && (
        <div className="mb-4 p-3 bg-red-100 text-red-700 rounded">
          {state.error}
        </div>
      )}

      <form action={formAction} className="space-y-4">
        <div>
          <label className="block text-sm font-medium mb-1">Name</label>
          <input
            type="text"

```

```

        name="name"
        className="w-full border rounded px-3 py-2"
        defaultValue="John Doe"
      />
      {state.errors?.name && (
        <p className="text-red-500 text-sm mt-1">{state.errors.name}</p>
      )}
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Email</label>
      <input
        type="email"
        name="email"
        className="w-full border rounded px-3 py-2"
        defaultValue="john@example.com"
      />
      {state.errors?.email && (
        <p className="text-red-500 text-sm mt-1">{state.errors.email}</p>
      )}
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Age</label>
      <input
        type="number"
        name="age"
        className="w-full border rounded px-3 py-2"
        defaultValue="25"
      />
      {state.errors?.age && (
        <p className="text-red-500 text-sm mt-1">{state.errors.age}</p>
      )}
    </div>

    <button
      type="submit"
      className="w-full bg-blue-500 text-white py-2 rounded hover:bg-blue-600"
    >
      Save Changes
    </button>
  </form>
</div>
);
}

```

useFormStatus for Loading State

```

// app/components/SubmitButton.tsx
'use client';

```

```
import { useFormStatus } from "react-dom";

export default function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button
      type="submit"
      disabled={pending}
      className="w-full bg-blue-500 text-white py-2 rounded hover:bg-blue-600 disabled
    >
      {pending ? "Saving..." : "Save Changes"}
    </button>
  );
}
```

4.2 API Routes (Route Handlers)

What are Route Handlers?

Route Handlers (`route.ts`) are API endpoints that run on the server.

Basic Route Handler

```
// app/api/posts/route.ts
import { NextResponse } from "next/server";

export async function GET() {
  const data = [
    { id: 1, title: "Post 1" },
    { id: 2, title: "Post 2" },
  ];

  return NextResponse.json(data);
}

export async function POST(request: Request) {
  const body = await request.json();

  // Save to database (example)
  console.log("Creating post:", body);

  return NextResponse.json(
    { success: true, data: body },
    { status: 201 }
  );
}
```

```
);
}
```

Route Handler with Parameters

```
// app/api/posts/[id]/route.ts
import { NextResponse } from "next/server";

interface RouteParams {
  params: Promise<{ id: string }>;
}

export async function GET(_request: Request, { params }: RouteParams) {
  const { id } = await params;

  // In real app, fetch from database
  const post = { id: Number(id), title: `Post ${id}`, body: "Content..." };

  if (!post) {
    return NextResponse.json(
      { error: "Post not found" },
      { status: 404 }
    );
  }

  return NextResponse.json(post);
}

export async function PUT(request: Request, { params }: RouteParams) {
  const { id } = await params;
  const body = await request.json();

  console.log(`Updating post ${id}:`, body);

  return NextResponse.json({ success: true, data: body });
}

export async function DELETE(_request: Request, { params }: RouteParams) {
  const { id } = await params;

  console.log(`Deleting post ${id}`);

  return NextResponse.json({ success: true });
}
```

Route Handler with Authentication

```
// app/api/protected/route.ts
import { NextResponse } from "next/server";
```

```
export async function GET(request: Request) {
  // In real app, check authentication
  const isAuthenticated = true;

  if (!isAuthenticated) {
    return NextResponse.json(
      { error: "Unauthorized" },
      { status: 401 }
    );
  }

  return NextResponse.json({ message: "Protected data" });
}
```

4.3 Authentication with Auth.js (NextAuth)

Installation

```
npm install @auth/core @auth/nextjs
```

Auth Configuration

```
// src/lib/auth.ts
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";
import GitHub from "next-auth/providers/github";

export const { handlers, signIn, signOut, auth } = NextAuth({
  providers: [
    GitHub({
      clientId: process.env.AUTH_GITHUB_ID!,
      clientSecret: process.env.AUTH_GITHUB_SECRET!,
    }),
    Credentials({
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" },
      },
      async authorize(credentials) {
        // In real app, verify against database
        if (
          credentials?.email === "test@example.com" &&
          credentials?.password === "password"
        ) {

```

```

        return {
          id: "1",
          name: "Test User",
          email: "test@example.com",
        };
      }
      return null;
    },
  )),
],
callbacks: {
  async session({ session, token }) {
    if (token && session.user) {
      session.user.id = token.sub as string;
    }
    return session;
  },
},
pages: {
  signIn: "/login",
  signOut: "/",
},
});

```

Auth API Routes

```

// app/api/auth/[...nextauth]/route.ts
import { handlers } from "@lib/auth";

export const { GET, POST } = handlers;

```

Middleware with Auth

```

// middleware.ts
import { NextResponse } from "next/server";
import { auth } from "@lib/auth";

export default auth((request) => {
  const isAuthenticated = !!request.auth;
  const path = request.nextUrl.pathname;

  const isProtectedRoute = path.startsWith("/dashboard");
  const isAuthRoute = path.startsWith("/login") || path.startsWith("/register");

  if (isProtectedRoute && !isAuthenticated) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  if (isAuthRoute && isAuthenticated) {

```

```

    return NextResponse.redirect(new URL("/dashboard", request.url));
  }

  return NextResponse.next();
});

export const config = {
  matcher: [
    "/((?!api|_next/static|_next/image|favicon.ico).*)",
  ],
};

```

Using Auth in Components

```

// app/dashboard/page.tsx
import { auth } from "@lib/auth";

export default async function DashboardPage() {
  const session = await auth();

  if (!session) {
    return <div>Please log in</div>;
  }

  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Dashboard</h1>
      <p>Welcome, {session.user?.name}!</p>
    </div>
  );
}

```

4.4 Optimistic Updates

useOptimistic Hook

For immediate UI feedback:

```

// app/posts/components/LikeButton.tsx
'use client';

import { useOptimistic } from "react";
import { toggleLike } from "@app/actions/posts";

interface LikeButtonProps {

```

```

    post: {
      id: string;
      likes: number;
      liked: boolean;
    };
  }

export default function LikeButton({ post }: LikeButtonProps) {
  const [optimisticState, addOptimisticLike] = useOptimistic(
    { likes: post.likes, liked: post.liked },
    (state, increment: number) => ({
      ...state,
      likes: state.likes + increment,
      liked: !state.liked,
    })
  );

  async function handleLike() {
    // Update optimistically
    addOptimisticLike(optimisticState.liked ? -1 : 1);

    // Actually update
    await toggleLike(post.id);
  }

  return (
    <button
      onClick={handleLike}
      className={`px-4 py-2 rounded ${
        optimisticState.liked
          ? "bg-red-500 text-white"
          : "bg-gray-200 text-gray-700"
      }`}
    >
      ❤️ {optimisticState.likes}
    </button>
  );
}

```

Day 4 Exercises

Exercise 4.1: Create a Server Action

Instructions:

1. Create a form that creates a new post
2. Use a Server Action for form submission

3. Validate the input with Zod

4. Show success/error states

```
// app/actions/posts.ts
'use server';

import { revalidatePath } from "next/cache";
import { z } from "zod";

const postSchema = z.object({
  title: z.string().min(3, "Title must be at least 3 characters"),
  body: z.string().min(10, "Body must be at least 10 characters"),
});

export interface PostFormState {
  success?: boolean;
  error?: string;
  errors?: {
    title?: string;
    body?: string;
  };
}

export async function createPost(
  prevState: PostFormState,
  formData: FormData
): Promise<PostFormState> {
  try {
    const validated = postSchema.parse({
      title: formData.get("title"),
      body: formData.get("body"),
    });

    console.log("Creating post:", validated);

    revalidatePath("/posts");
    return { success: true };
  } catch (error) {
    if (error instanceof z.ZodError) {
      const fieldErrors: Record<string, string> = {};
      error.errors.forEach((err) => {
        if (err.path[0]) {
          fieldErrors[err.path[0] as string] = err.message;
        }
      });
      return { success: false, errors: fieldErrors };
    }
    return { success: false, error: "Something went wrong" };
  }
}
```

```
// app/posts/new/page.tsx
'use client';

import { useFormState } from "react-dom";
import { createPost } from "@app/actions/posts";

const initialState = {
  success: false,
  error: "",
  errors: {},
};

export default function NewPostPage() {
  const [state, formAction] = useFormState(createPost, initialState);

  return (
    <div className="p-8 max-w-2xl mx-auto">
      <h1 className="text-3xl font-bold mb-6">Create New Post</h1>

      {state.success && (
        <div className="mb-4 p-3 bg-green-100 text-green-700 rounded">
          Post created successfully!
        </div>
      )}

      {state.error && (
        <div className="mb-4 p-3 bg-red-100 text-red-700 rounded">
          {state.error}
        </div>
      )}

      <form action={formAction} className="space-y-4">
        <div>
          <label className="block text-sm font-medium mb-1">Title</label>
          <input
            type="text"
            name="title"
            className="w-full border rounded px-3 py-2"
          />
          {state.errors?.title && (
            <p className="text-red-500 text-sm mt-1">{state.errors.title}</p>
          )}
        </div>

        <div>
          <label className="block text-sm font-medium mb-1">Body</label>
          <textarea
            name="body"
            rows={6}
            className="w-full border rounded px-3 py-2"
          />
          {state.errors?.body && (
```

```

        <p className="text-red-500 text-sm mt-1">{state.errors.body}</p>
      )}
    </div>

    <button
      type="submit"
      className="bg-blue-500 text-white px-6 py-2 rounded hover:bg-blue-600"
    >
      Create Post
    </button>
  </form>
</div>
);
}

```

Exercise 4.2: Create an API Route

Instructions:

1. Create an API route at `/api/posts`
2. Implement GET, POST, PUT, DELETE methods
3. Use in-memory data store

```

// app/api/posts/route.ts
import { NextResponse } from "next/server";

let posts = [
  { id: 1, title: "Post 1", body: "Content 1" },
  { id: 2, title: "Post 2", body: "Content 2" },
];

export async function GET() {
  return NextResponse.json(posts);
}

export async function POST(request: Request) {
  const body = await request.json();
  const newPost = {
    id: posts.length + 1,
    title: body.title,
    body: body.body,
  };
  posts.push(newPost);
  return NextResponse.json(newPost, { status: 201 });
}

// app/api/posts/[id]/route.ts
import { NextResponse } from "next/server";

```

```

let posts = [
  { id: 1, title: "Post 1", body: "Content 1" },
  { id: 2, title: "Post 2", body: "Content 2" },
];

interface RouteParams {
  params: Promise<{ id: string }>;
}

export async function PUT(request: Request, { params }: RouteParams) {
  const { id } = await params;
  const body = await request.json();
  const index = posts.findIndex((p) => p.id === Number(id));

  if (index === -1) {
    return NextResponse.json({ error: "Not found" }, { status: 404 });
  }

  posts[index] = { ...posts[index], ...body };
  return NextResponse.json(posts[index]);
}

export async function DELETE(_request: Request, { params }: RouteParams) {
  const { id } = await params;
  posts = posts.filter((p) => p.id !== Number(id));
  return NextResponse.json({ success: true });
}

```

Exercise 4.3: Implement Authentication

Instructions:

1. Set up Auth.js with GitHub provider
2. Add email/password credentials provider
3. Protect dashboard routes
4. Add login/logout functionality

Day 4 Summary

Key Takeaways

1. **Server Actions:** Run on server, type-safe, handle form submissions.
2. **Route Handlers:** `route.ts` files for API endpoints.
3. **Authentication:** Use Auth.js/NextAuth for auth providers.

4. **Middleware:** Protect routes and handle redirects.

5. **Optimistic Updates:** Use `useOptimistic` for immediate UI feedback.

Day 5: Performance, Deployment & Production Readiness

Learning Objectives

By the end of Day 5, you will be able to:

- Optimize Next.js applications for performance
- Use `next/image` for image optimization
- Configure SEO and metadata
- Deploy to Vercel
- Implement production monitoring and logging

5.1 Performance Optimization

Image Optimization

The `next/image` component provides automatic image optimization:

```
import Image from "next/image";

export default function HeroImage() {
  return (
    <Image
      src="/hero.jpg"
      alt="Hero"
      width={1200}
      height={600}
      priority // Preload above the fold
      className="rounded-lg"
    />
  );
}
```

Font Optimization

```
// app/layout.tsx
import { Inter, Roboto_Mono } from "next/font/google";

const inter = Inter({
  subsets: ["latin"],
  display: "swap", // FOIT prevention
});

const robotoMono = Roboto_Mono({
  subsets: ["latin"],
  variable: "--font-roboto-mono",
});

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en" className={`${inter.className} ${robotoMono.variable}`}>
      <body>{children}</body>
    </html>
  );
}
```

Script Optimization

```
import Script from "next/script";

export default function Analytics() {
  return (
    <>
      <Script
        src="https://analytics.example.com/script.js"
        strategy="lazyOnload" // Don't block rendering
      />
      <Script
        id="inline-script"
        strategy="afterInteractive"
        dangerouslySetInnerHTML={{
          __html: `console.log("Loaded");`,
        }}
      />
    </>
  );
}
```

5.2 Bundle Optimization

Analyzing Bundle Size

```
npm install @next/bundle-analyzer -D
```

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  experimental: {
    // Enable Turbopack for faster builds
  },
};

export default nextConfig;
```

Dynamic Imports

For lazy loading components:

```
'use client';

import dynamic from "next/dynamic";

// Component will be loaded only when needed
const HeavyComponent = dynamic(
  () => import("@/components/HeavyComponent"),
  {
    loading: () => <p>Loading...</p>,
    ssr: false, // Don't server render
  }
);

export default function Page() {
  return (
    <div>
      <h1>Page</h1>
      <HeavyComponent />
    </div>
  );
}
```

5.3 SEO and Metadata

Open Graph and Social Sharing

```
// app/blog/[slug]/page.tsx
import type { Metadata } from "next";

export async function generateMetadata({
  params,
}: {
  params: Promise<{ slug: string }>;
}): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPost(slug);

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
      url: `https://myapp.com/blog/${slug}`,
      siteName: "My Next.js App",
      images: [
        {
          url: `https://myapp.com/api/og?title=${encodeURIComponent(post.title)}`,
          width: 1200,
          height: 630,
        },
      ],
      type: "article",
      publishedTime: post.date,
    },
    twitter: {
      card: "summary_large_image",
      title: post.title,
      description: post.excerpt,
    },
  };
}
```

Dynamic OG Image Generation

```
// app/api/og/route.ts
import { ImageResponse } from "next/og";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const title = searchParams.get("title") || "Default Title";

  return new ImageResponse(
    (
```

```
    <div
      style={{
        fontSize: 60,
        background: "#1a1a2e",
        color: "white",
        padding: 50,
        width: 1200,
        height: 630,
        display: "flex",
        alignItems: "center",
        justifyContent: "center",
      }}
    >
      {title}
    </div>
  ),
  {
    width: 1200,
    height: 630,
  }
);
}
```

5.4 Deployment

Deploying to Vercel

Step 1: Push code to GitHub

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/yourusername/your-repo.git
git push -u origin main
```

Step 2: Deploy on Vercel

1. Go to vercel.com
2. Import GitHub repository
3. Configure environment variables
4. Deploy

Environment Variables

```
# .env.local (local development)
DATABASE_URL="postgresql://..."
AUTH_SECRET="..."
AUTH_GITHUB_ID="..."
AUTH_GITHUB_SECRET="..."
```

Production Build

```
# Build production bundle
npm run build

# Run production server
npm run start
```

5.5 Monitoring and Logging

Next.js 15 Production Features

Feature	Description
Turbopack	76.7% faster startup, 96.3% faster Fast Refresh
PPR	Partial Prerendering (experimental)
Edge Runtime	Faster, CDN-based execution

Logging in Production

```
// lib/logger.ts
export function logInfo(message: string, data?: Record<string, unknown>) {
  if (process.env.NODE_ENV === "development") {
    console.log(` [INFO] ${message}`, data);
  }
  // In production, send to logging service
}

export function logError(message: string, error?: Error) {
  if (process.env.NODE_ENV === "development") {
    console.error(` [ERROR] ${message}`, error);
  }
  // In production, send to error tracking
}
```

Day 5 Exercises

Exercise 5.1: Optimize Images

Instructions:

1. Add an image to your home page
2. Use `next/image` with proper sizing
3. Add priority for hero image

```
// app/page.tsx
import Image from "next/image";
import heroImage from "@public/hero.jpg";

export default function HomePage() {
  return (
    <main className="min-h-screen">
      <div className="relative h-[500px] w-full">
        <Image
          src={heroImage}
          alt="Hero"
          fill
          priority
          className="object-cover"
        />
        <div className="absolute inset-0 flex items-center justify-center">
          <h1 className="text-4xl font-bold text-white">Welcome!</h1>
        </div>
      </div>
    </main>
  );
}
```

Exercise 5.2: Add Open Graph Metadata

Instructions:

1. Add Open Graph metadata to the blog post page
2. Create an OG image generation API route

Exercise 5.3: Deploy to Vercel

Instructions:

1. Push your code to GitHub
2. Import to Vercel
3. Deploy and verify

Day 5 Summary

Key Takeaways

1. **Image Optimization:** Use `next/image` for automatic optimization.
2. **Font Optimization:** Use Next.js font system.
3. **Bundle Analysis:** Analyze and optimize bundle size.
4. **SEO:** Use `generateMetadata` for dynamic SEO.
5. **Deployment:** Vercel is the recommended deployment platform.

Appendix: Answer Keys & Code Solutions

Day 1 Solutions

Exercise 1.1: Pages

✅ Contact page created ✅ Blog page with 5 posts ✅ Dynamic blog post page

Exercise 1.2: Layouts

✅ Root layout with navigation and footer ✅ Blog layout with sidebar

Exercise 1.3: Client Component

✅ Counter component with `useState`

Exercise 1.4: Metadata

✅ About page metadata ✅ Blog post dynamic metadata

Day 2 Solutions

Exercise 2.1: Dynamic Products

✅ Products page with list ✅ Dynamic product page with `generateStaticParams`

Exercise 2.2: Route Groups

✅ Marketing group with homepage and about ✅ Dashboard group with dashboard page ✅ Auth group with login and register

Exercise 2.3: Middleware

✅ Authentication middleware ✅ Redirect unauthenticated users ✅ Redirect authenticated users from auth routes

Exercise 2.4: Loading/Error

✅ Loading page ✅ Error page with retry ✅ Not found page

Day 3 Solutions

Exercise 3.1: Data Fetching

✅ Posts page with API fetch ✅ Display posts with title and body

Exercise 3.2: Caching

✅ `force-cache` strategy ✅ `no-store` strategy ✅ `revalidate` strategy

Exercise 3.3: Streaming

✅ Suspense with fallback ✅ Independent loading for each section

Exercise 3.4: Parallel Routes

✅ `@analytics` slot ✅ `@metrics` slot ✅ Independent loading

Day 4 Solutions

Exercise 4.1: Server Action

✅ Create post action with Zod validation ✅ Form with `useFormState` ✅ Success/error states

Exercise 4.2: API Routes

✅ GET /api/posts ✅ POST /api/posts ✅ PUT /api/posts/[id] ✅ DELETE /api/posts/[id]

Exercise 4.3: Authentication

✅ Auth.js configuration ✅ GitHub provider ✅ Credentials provider ✅ Protected routes

Day 5 Solutions

Exercise 5.1: Image Optimization

✅ next/image implementation ✅ Priority for hero image

Exercise 5.2: Open Graph

✅ generateMetadata with OG tags ✅ OG image API route

Exercise 5.3: Deployment

✅ GitHub push ✅ Vercel import ✅ Production deployment

Appendix: Project Setup Guide

Initial Setup Commands

```
# Create Next.js 15 project
npx create-next-app@latest nextjs-course-app \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir \
  --import-alias @/*

cd nextjs-course-app

# Install dependencies
npm install @types/node @types/react @types/react-dom -D
npm install next @next/bundle-analyzer
npm install @auth/core @auth/nextjs
npm install zod
npm install drizzle-orm @neondatabase/serverless
```

Environment Variables

```
# .env.local
DATABASE_URL="postgresql://..."
AUTH_SECRET="your-secret"
AUTH_GITHUB_ID="your-id"
AUTH_GITHUB_SECRET="your-secret"
NEXT_PUBLIC_API_URL="http://localhost:3000"
```

Project Structure

```
src/
├── app/
│   ├── (marketing)/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   └── about/
│   │       └── page.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx
│   │   └── dashboard/
│   │       └── page.tsx
│   ├── (auth)/
│   │   ├── layout.tsx
│   │   ├── login/
│   │   │   └── page.tsx
│   │   └── register/
│   │       └── page.tsx
│   ├── api/
│   │   ├── auth/[...nextauth]/
│   │   │   └── route.ts
│   │   └── posts/
│   │       ├── route.ts
│   │       └── [id]/
│   │           └── route.ts
│   ├── blog/
│   │   ├── page.tsx
│   │   ├── [slug]/
│   │   │   └── page.tsx
│   │   ├── loading.tsx
│   │   └── error.tsx
│   ├── components/
│   ├── lib/
│   ├── globals.css
│   ├── layout.tsx
│   └── not-found.tsx
├── middleware.ts
└── next.config.ts
```

```
|— tailwind.config.ts
|— tsconfig.json
```

END OF FACILITATOR EDITION

Key Citations:

- App Router and Server Components fundamentals
- Dynamic routes and route groups
- Middleware implementation
- Parallel routes and advanced patterns
- Server Actions and form handling
- Full-stack course structure